

```
irb(main):096> Object.is_a?(BasicObject)
=> true...
```

This happens because you are checking the relationships of **Class objects** themselves, not standard instances.

In Ruby, every class is also an object (an instance of the class `Class`). Because of this dual nature, their relationships form a deliberate, closed loop in the object model. 🔗

Here is the breakdown of why both statements evaluate to `true` .

1. Why `Object.is_a?(BasicObject)` is true

The `.is_a?` method checks if an object is an instance of a given class or any of its superclasses. 🔗

- `Object` is a class.
- `BasicObject` is the root superclass of `Object` .
- Therefore, `Object` inherits from `BasicObject` .
- Since `Object` inherits from `BasicObject` , any instance of `Object` (including the `Object` class itself) is a type of `BasicObject` . 🔗

2. Why `BasicObject.is_a?(Object)` is true

This is the part that feels contradictory, but it makes sense when you look at what `BasicObject` actually is.

- `BasicObject` is a class, which means it is an instance of the class `Class` (`BasicObject.class # => Class`).
- The class `Class` inherits from `Module` , which inherits from `Object` .
- Because `Class` inherits from `Object` , **every class object in Ruby is an instance of `Object`** .
- Since `BasicObject` is a class object, it is fundamentally an instance of `Object` . 🔗

The Visual Loop

To see the difference, you must separate **inheritance** (superclass) from **instantiation** (class):

- Inheritance Hierarchy (The Ancestors):**
`Class < Module < Object < BasicObject`
(Here, *BasicObject* is at the absolute top).
- Instantiation Hierarchy (What they are):**
`Object` is a `Class` .
`BasicObject` is a `Class` .
`Class` is a `Class` . 🔗

Because `Class` inherits from `Object` , `BasicObject` ends up wearing two hats: it is the **parent** of `Object` in terms of inheritance, but it is a **child** of `Object` in terms of its data type (because it is a `Class` object). 🔗

How to verify the "True" Top Level

If you want to look strictly at the inheritance tree without involving the `Class` instantiation loop, use `.ancestors` or `<` instead:

ruby

```
# Check the strict inheritance chain
Object.ancestors      # => [Object, Kernel, BasicObject]
BasicObject.ancestors # => [BasicObject]

# Check if one is a subclass of the other
Object < BasicObject   # => true  (Object inherits from BasicObject)
BasicObject < Object   # => false (BasicObject does NOT inherit from Object)
```

Use code with caution.



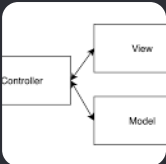
To help clear up any remaining confusion, let me know if you want to explore how the **Class** and **Module** classes fit into this loop, or if you want to see how this affects **method lookup** in Ruby.

16 sites



53 Ruby on Rails Interview Questions and Answers | by GreekDataGuy |...

2 Apr 2020 — 2. Everything (almost) is an object in Ruby. Please explain In object-...



Medium



Rails Macros and Metaprogramming - Learn Ruby on Rails

We know that everything in Ruby is an object. Even classes are objects since they are instances of the `Class` class. So i...

BigBinary Academy



Object Oriented Ruby and Inheritance

`is_a?(Class)` With `is_a?` you can check if the class or one of the superclasses (base) of an object. Try this out. Before...

codebar - Tutorials



Show all